Department of Computer Science and Engineering

A PROJECT REPORT ON

RL GameAgent: A Modular Deep Q-Network Family for Classic Control Games

Submitted in partial fulfilment of the requirements for

Course Code: 23CSE401

Subject: Foundations of Artificial Intelligence

Submitted by

Shyam Tripathi

Roll No.: CH.SC.U4CSE23160

Under the guidance of

Dr. J Umamageswaran

Assistant Professor Selection Grade

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF COMPUTING

CHENNAI – 601103

October 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING CHENNAI

CERTIFICATE

This is to certify that the project report entitled **"RL GAMEAGENT: A MODULAR DEEP Q-NETWORK FAMILY FOR CLASSIC CONTROL GAMES"** is a bona fide record of the work carried out by **Shyam Tripathi (CH.SCU4CSE23160)** under my supervision and guidance, in partial fulfilment of the requirements for the course **23CSE401** – **Foundations of Artificial Intelligence** of **Amrita School of Computing, Chennai** during the academic year **2026**.

The work embodied in this report has not been submitted to any other institution or university for the award of any degree or diploma.

Mentor
Dr. J Umamagewaran
Assistant Professor Selection Grade

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project mentor, **Dr. J Umamagewaran,**

for their continuous guidance, patience and valuable suggestions throughout the development of this project. Their feedback on the experimental design and the report structure greatly improved the quality of the work.

I am thankful to the Head of the Department, **Dr. S. Bhagvathi Priya**, and all the faculty members of **Computer Science and Engineering** for providing the academic environment and resources needed to complete this project. I also thank the laboratory staff for their support with the computing facilities.

Finally, I thank my family and friends for their encouragement and understanding during the course of this work. Their support kept me motivated through the long training runs and debugging sessions.

Shyam Tripathi
Roll No.: CH.SC.U4CSE23160

ABSTRACT

Deep reinforcement learning has produced agents that learn control policies directly from raw experience, but the research literature contains many extensions to the original Deep Q-Network (DQN), and it is often unclear how much each one contributes in practice. This project presents RL GameAgent, a modular DQN framework in which each well-known extension can be switched on or off through a configuration preset. The extensions implemented are Double Q-learning, the dueling network architecture, Prioritized Experience Replay with a sum-tree, and n-step returns with per-transition discounting.

The framework is evaluated on two classic-control environments from Gymnasium, CartPole-v1 and Acrobot-v1. Training runs are tracked through a typed configuration object saved with every checkpoint, and model selection is based on fixed-seed greedy evaluation rather than noisy training rewards. An ablation runner trains each variant over several random seeds and produces an aggregate comparison table. A Streamlit dashboard provides live training charts, a playback view of saved agents, and analytics across runs.

The results show how solve rate, sample efficiency and final evaluation score change as extensions are added. The project also demonstrates practical engineering choices that make reinforcement learning experiments reproducible and easy to inspect.

Keywords: Deep Q-Network, Double DQN, Dueling architecture, Prioritized Experience Replay, n-step returns, reinforcement learning, ablation study.

CONTENTS

1. Introduction 2
2. Background and Related Work 3
3. System Design and Architecture 3
4. Implementation 4
5. Experimental Setup 5
6. Results and Discussion 6
7. Conclusion 7
8. Future Work 7
9. References 8

LIST OF FIGURES

Figure 1: System architecture 3
Figure 2: Dashboard overview page 5
Figure 3: Training page with live charts 5
Figure 4: Learning curve of a training run 6
Figure 5: Ablation summary table 7

1. INTRODUCTION

1.1 Background

Reinforcement learning (RL) studies how an agent should act in an environment so as to maximise cumulative reward. Unlike supervised learning, the agent is never told the correct action; it must discover good behaviour through trial and error. Value-based methods estimate the expected return of taking an action in a given state, and the agent chooses actions that appear most valuable. Q-learning is the classic example of this approach.

When the state is described by a continuous vector rather than a small table, the Q-function is approximated by a neural network. The Deep Q-Network (DQN) of Mnih et al. (2015) made this practical by adding two stabilising ideas: an experience replay buffer that breaks correlations between consecutive samples, and a periodically updated target network that provides more stable regression targets. Since then many refinements have been proposed, each addressing a specific weakness of the original algorithm.

1.2 Problem Statement

The refinements of DQN are usually published as separate papers, each evaluated on its own benchmark with its own training setup. Students and practitioners therefore find it difficult to answer a simple question: which of these ideas actually helps, on which problems, and by how much? Answering it requires implementations that differ only in the feature being studied, run under identical conditions and across several random seeds.

1.3 Objectives

- To implement a DQN baseline and four progressively richer variants in one shared code base.
- To make each extension (Double DQN, dueling head, prioritized replay and n-step returns) switchable through a preset, so that ablations differ in exactly one factor.
- To evaluate the variants on CartPole-v1 and Acrobot-v1 with multiple seeds and report aggregate statistics.
- To provide a dashboard for training, playback and analysis, and a command-line interface for batch experiments.

1.4 Scope

The project is restricted to environments with vector observations and discrete action spaces. Image-based (CNN) agents and continuous-control algorithms are outside the scope of this work and are discussed as future work in Chapter 8.

2. BACKGROUND AND RELATED WORK

2.1 Q-Learning and the Bellman Target

The optimal action-value function satisfies the Bellman optimality equation. In DQN, a network with parameters θ is trained to minimise the difference between its prediction $Q(s, a; \theta)$ and the target $y = r + \gamma \max_{a'} Q(s', a'; \theta^{-})$, where θ^{-} are the parameters of a slowly updated

target network and γ is the discount factor. The loss is usually a Huber function, which is less sensitive to outliers than squared error.

2.2 Extensions Implemented in This Project

Table 1 summarises the four extensions included in the framework, together with the reference paper for each one and the module in which it is implemented.

Table 1: Extensions implemented in RL GameAgent

Extension	Reference	Idea	Module
DQN	Mnih et al., 2015	Replay buffer and target network	agent.py
Double Q-learning	van Hasselt et al., 2016	Online net selects the action, target net evaluates it	agent.py
Dueling architecture	Wang et al., 2016	Separate value V(s) and advantage A(s,a) streams	networks.py
Prioritized replay	Schaul et al., 2016	Sample surprising transitions more often, with IS weights	replay_buffer.py
N-step returns	Sutton, 1988	Propagate reward over n steps with discount γ^n	replay_buffer.py

3. SYSTEM DESIGN AND ARCHITECTURE

The system is organised into modules with clearly separated responsibilities. The core logic for replay buffers, configuration and utilities depends only on NumPy, so it can be tested quickly without loading PyTorch. The neural network and agent depend on PyTorch, and the user interfaces depend on the training and evaluation modules. Figure 1 shows the overall architecture.

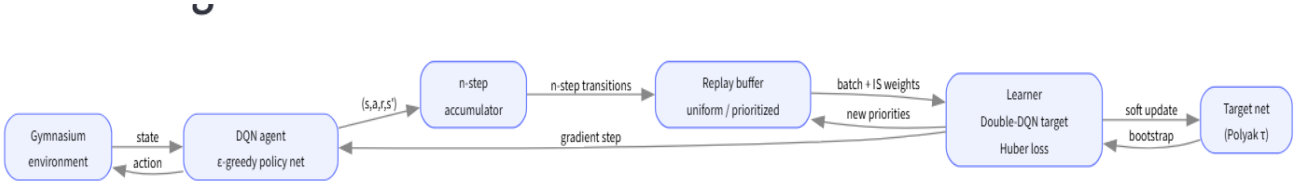


Figure 1: System architecture of RL GameAgent

3.1 Configuration Layer

A single configuration layer, configs.py, is the source of truth for paths, game specifications and algorithm variants. Each game is described by a GameSpec record that holds the environment identifier, the solved threshold and default hyper-parameters. Each variant is a set of boolean and integer flags. A typed TrainConfig dataclass merges these in a fixed order: defaults, then game specification, then variant flags, then user overrides. The complete configuration is stored with every run and every checkpoint, so results can be reproduced and checkpoints remain self-describing.

3.2 Network Architecture

The Q-network in networks.py is a fully connected network with two hidden layers of 128 ReLU units by default. In the plain configuration it ends in a single linear head that outputs one Q-value per action. In the dueling configuration the shared features feed two streams, one producing a scalar state value V(s) and the other producing advantages A(s, a) for each action. These are combined as Q(s, a) = V(s) + A(s, a) − mean over a' of A(s, a'), which keeps the decomposition identifiable.

3.3 Replay and Learning Layer

The replay layer stores transitions in preallocated NumPy arrays. An n-step accumulator converts one-step transitions into n-step transitions, storing the discounted reward sum and the discount factor γ^k that applies to the bootstrap. When prioritized replay is enabled, a sum-tree gives O(log N) proportional sampling and priority updates. The agent then performs one gradient step per environment step once the buffer has warmed up.

3.4 Run Tracking and Model Selection

Each training run writes three artefacts: a per-episode CSV log, a JSON file with configuration, status and summary, and model checkpoints. Model selection is deliberately honest. The best checkpoint is chosen by greedy evaluation over a fixed set of seeds, not by the ε-greedy training reward, which is noisy and biased by exploration. The reported final score is computed by a separate evaluation of that checkpoint on different seeds. Writes to the JSON registry are atomic, so the dashboard never reads a partially written file.

4. IMPLEMENTATION

The project is written in Python 3.10 or later, using PyTorch, Gymnasium, NumPy, pandas, Streamlit and pytest. Training runs on the GPU when CUDA is available and on the CPU otherwise.

4.2 Acting and Exploration

During training the agent follows an ε-greedy policy. With probability ε it selects a random action, and otherwise the action with the highest predicted Q-value. ε starts at 1.0 and decays geometrically once per episode, with a floor of 0.01. The decay rate differs by game: 0.99 for CartPole-v1 and 0.98 for Acrobot-v1. Greedy evaluation is used for all measurements that matter for model selection.

4.3 Learning Step

Each learning step samples a mini-batch of 64 transitions. For Double DQN, the online network selects the best next action and the target network evaluates it, which reduces the over-estimation that occurs when a single network both selects and scores actions. The loss is the mean of importance-weighted Huber errors, gradients are clipped to a norm of 10, and the optimiser is Adam. The target network is updated after every step by Polyak averaging with coefficient $\tau = 0.005$.

Termination and truncation are handled separately. Only true termination stops bootstrapping. A time-limit truncation, such as the 500-step limit in CartPole, is not a real end of the task, so the value of the final state is still bootstrapped. This is a common source of bias in simple DQN implementations.

4.4 Prioritized Replay

Transitions are sampled with probability proportional to p_i^{α}, where p_i is the absolute TD error plus a small constant and $\alpha = 0.6$. Importance-sampling weights $w_i = (N \cdot P(i))^{(-\beta)}$ correct the resulting bias. β starts at 0.4 and is annealed linearly to 1.0 over training. New transitions receive the current maximum priority, which guarantees that each is replayed at least once.

4.5 Command-Line Tools

Four command-line entry points are provided. train.py runs a single experiment. play.py loads a checkpoint and either opens a window to render episodes or evaluates the policy without rendering and prints summary statistics. compare.py runs every requested variant over several seeds and writes an aggregate table to the logs directory. pytest runs the unit and smoke tests.

4.6 Streamlit Dashboard

The dashboard is organised into four pages. The Overview page shows the architecture, a component table and the ablation presets. The Train page starts training in a background thread, so the interface stays responsive, and shows live charts with a Stop button. The Play page renders any saved checkpoint frame by frame. The Analytics page provides a leaderboard, the aggregated ablation table, comparison curves across runs, and per-run diagnostics for loss, Q-values, ε and the PER β parameter.

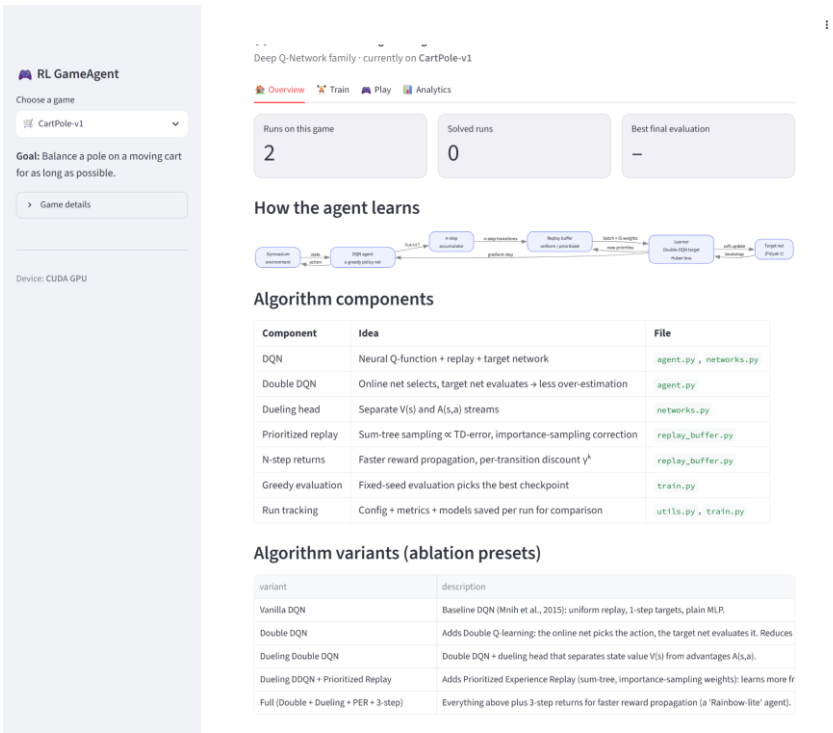


Figure 2: Dashboard overview page



Figure 3: Training page with live charts

5. EXPERIMENTAL SETUP

5.1 Hyper-parameters

Table 2 lists the main hyper-parameters used for all experiments. Game-specific values are shown in separate columns where they differ.

Table 2: Main hyper-parameters used in the experiments

Parameter	CartPole-v1	Acrobot-v1
Training episodes (budget)	300	200
Learning rate (Adam)	1×10^{-3}	5×10^{-4}
Exploration decay per episode	0.99	0.98
Solved threshold	Mean reward ≥ 475 over 100 episodes	Mean reward ≥ -100 over 100 episodes
Discount factor γ	0.99	0.99
Batch size / replay capacity	64 / 50,000	64 / 50,000
Warm-up steps	1,000	1,000
Hidden layers	128 × 128 (ReLU)	128 × 128 (ReLU)
Target update coefficient τ	0.005	0.005
Gradient clipping (norm)	10	10
PER α / β (start → end)	0.6 / 0.4 → 1.0	0.6 / 0.4 → 1.0
Greedy evaluation	Every 10 episodes, 5 runs	Every 10 episodes, 5 runs

Parameter	CartPole-v1	Acrobot-v1
Final evaluation	20 episodes, separate seeds	20 episodes, separate seeds

5.2 Variants Compared

Five presets form the ablation ladder: plain DQN, Double DQN, Dueling Double DQN, Dueling Double DQN with prioritized replay, and the full agent that adds 3-step returns. Each preset adds exactly one feature to the previous one, so the change in performance between adjacent rows can be attributed to that feature.

5.3 Evaluation Protocol

Each variant is trained with several seeds, starting from a base seed of 42. Because single runs of DQN vary considerably, results are reported as means and standard deviations across seeds rather than as the outcome of one run. The metrics recorded for each run are the episode at which the solved threshold was first reached, the solve rate across seeds, the final greedy evaluation score, and the wall-clock time.

6. RESULTS AND DISCUSSION

This chapter presents the outputs of the framework. The numerical values in the tables below are to be filled in from the ablation runs in the logs directory. The figures show the corresponding screens of the dashboard.

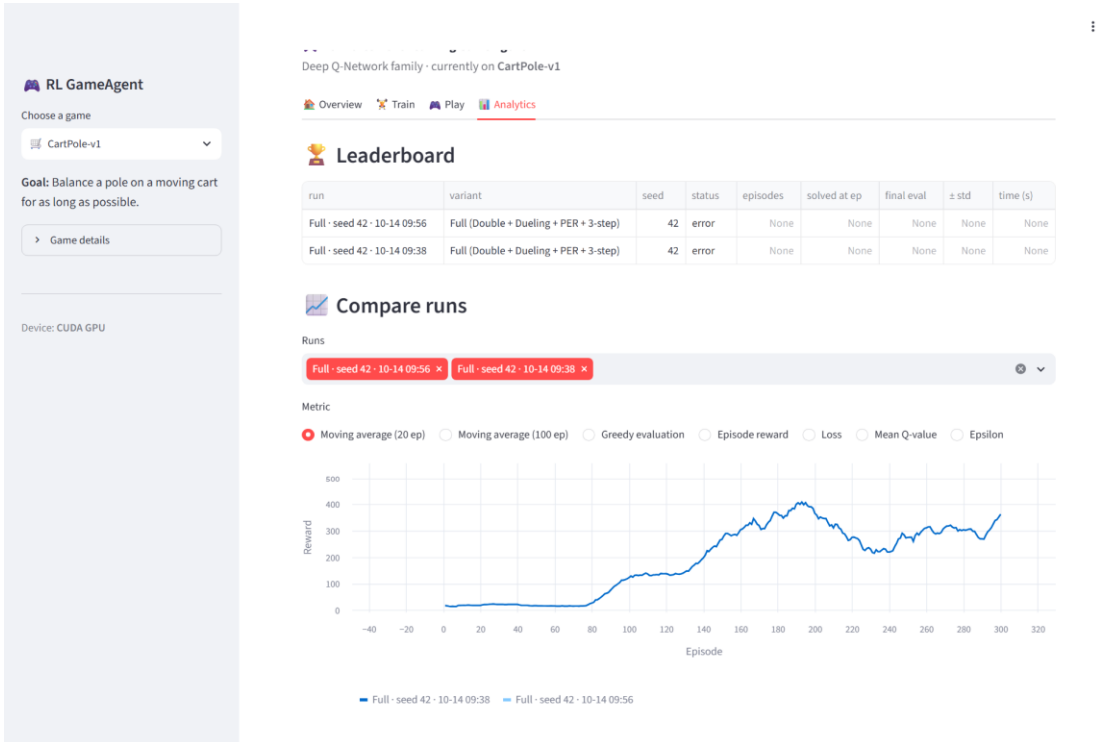




Figure 4: Learning curve of a training run

6.1 Ablation Results

Table 3 gives the aggregate results of the ablation study on CartPole-v1, produced by compare.py.

Table 3: Ablation results on CartPole-v1 (mean over seeds)

variant	runs	solved_rate	episodes_to_solve	final_eval_mean	final_eval_std	seconds
dqn	3	0.0	NaN	306.40	170.12	98.45
ddqn	3	0.0	NaN	411.88	152.62	107.55
dueling_ddqn	3	0.0	NaN	491.60	14.55	111.37
per_dueling_ddqn	3	0.0	NaN	481.32	32.36	135.86
full	3	0.0	NaN	492.45	13.08	262.70

Figure 5: Ablation summary table in the dashboard

6.2 Discussion

Based on the ablation table, the discussion should address three questions. First, does Double DQN reduce the variance between seeds? Second, does the dueling head improve sample efficiency, measured as the number of episodes needed to reach the solved threshold? Third, does prioritized replay or 3-step returns change the final score, and at what cost in training time? Differences smaller than the standard deviation across seeds should be described as inconclusive rather than as improvements.

7. CONCLUSION

This project developed RL GameAgent, a modular framework that implements the core extensions of the Deep Q-Network family behind a common interface. Each extension can be enabled through a preset, which makes controlled ablation studies straightforward. The framework supports training, evaluation, playback and analysis from both the command line and an interactive dashboard.

The project also emphasises correctness and reproducibility. Termination is distinguished from truncation, model selection uses fixed-seed evaluation, and every run stores its complete configuration. These choices help ensure that reported differences reflect the algorithm rather than measurement noise.

The main lesson from the experiments is that single runs of deep reinforcement learning are unreliable indicators of performance, and that multiple seeds are necessary before any extension can be credited with an improvement.

8. FUTURE WORK

Several extensions to the project are possible:

- Noisy networks for exploration in place of ε-greedy action selection.
- A distributional (C51) value head, which models the full distribution of returns.
- Multi-environment training, so that one agent can be trained on several games.
- Convolutional encoders for pixel-based observations such as Atari games.
- Continuous-action algorithms such as DDPG or SAC for control tasks like MuJoCo environments.
- Hyper-parameter search with an automated tool, to reduce manual tuning.

9. REFERENCES

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015.

[2] H. van Hasselt, A. Guez and D. Silver, “Deep reinforcement learning with double Q-learning,” in Proc. AAAI Conference on Artificial Intelligence, 2016.

[3] Z. Wang et al., “Dueling network architectures for deep reinforcement learning,” in Proc. International Conference on Machine Learning (ICML), 2016.

[4] T. Schaul, J. Quan, I. Antonoglou and D. Silver, “Prioritized experience replay,” in Proc. International Conference on Learning Representations (ICLR), 2016.

[5] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.

[6] M. Towers et al., “Gymnasium: A standard interface for reinforcement learning environments,” 2023. [Online]. Available: https://gymnasium.farama.org